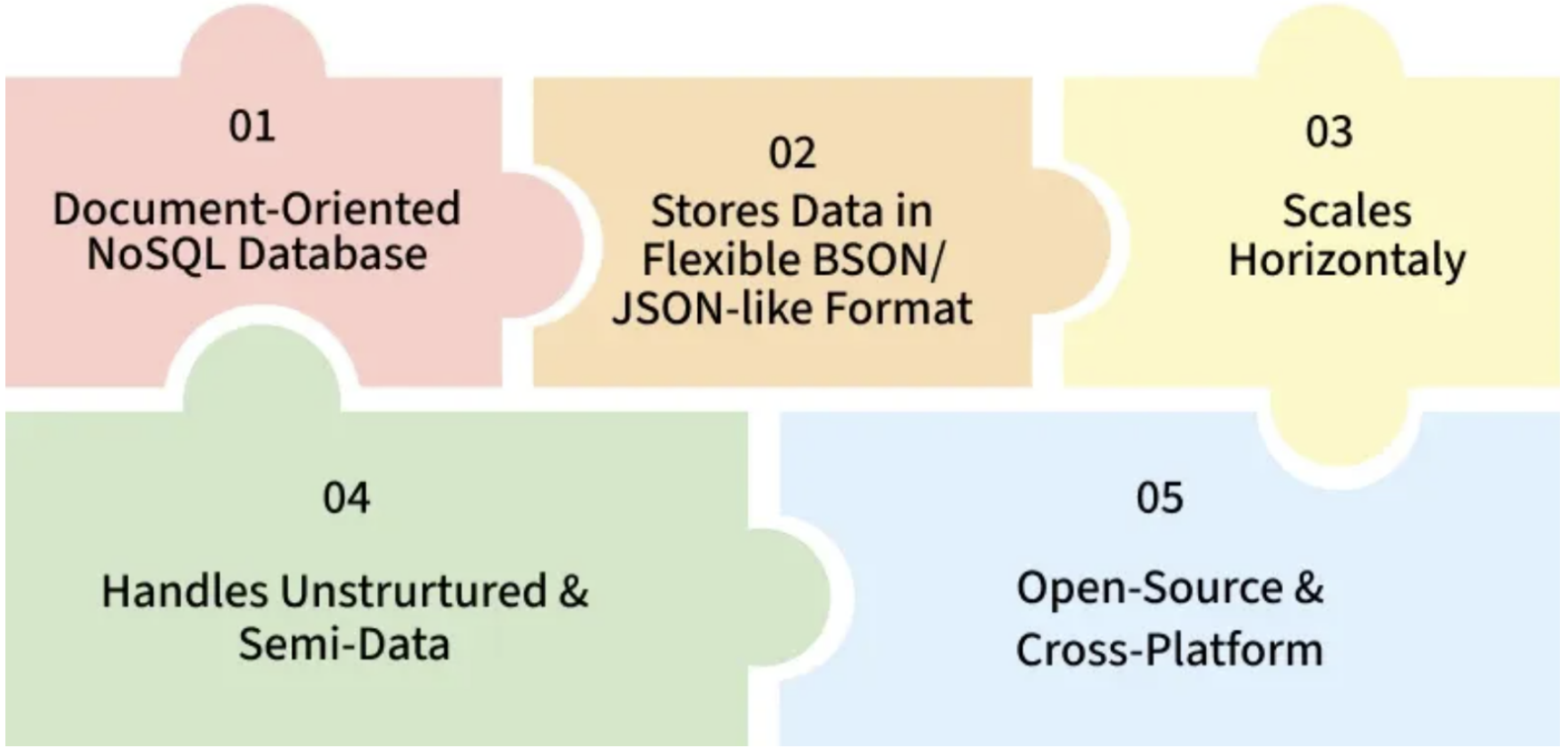
24CSA212 – Full Stack Frameworks

INTRODUCTION TO MONGODB

MongoDB Introduction

The SSPL is a sourceavailable license created by MongoDB that requires anyone offering the software as a service to release the entire source code of the service stack.
The **Server Side Public License (SSPL)** was introduced by MongoDB in 2018 as a modification of the GNU Affero General Public License (AGPL v3) to address the use of software over a network.

- MongoDB is a popular document-oriented NoSQL database that stores data in a flexible BSON format, enabling fast and efficient storage and retrieval.
- It is available under the SSPL, which is not recognized as an open-source license by the Open Source Initiative due to commercial use restrictions.



- Stores large volumes of data efficiently.
- Uses a document-based model instead of tables.
- Classified as a NoSQL (non-relational) database.
- Data is stored in BSON format, similar to JSON.

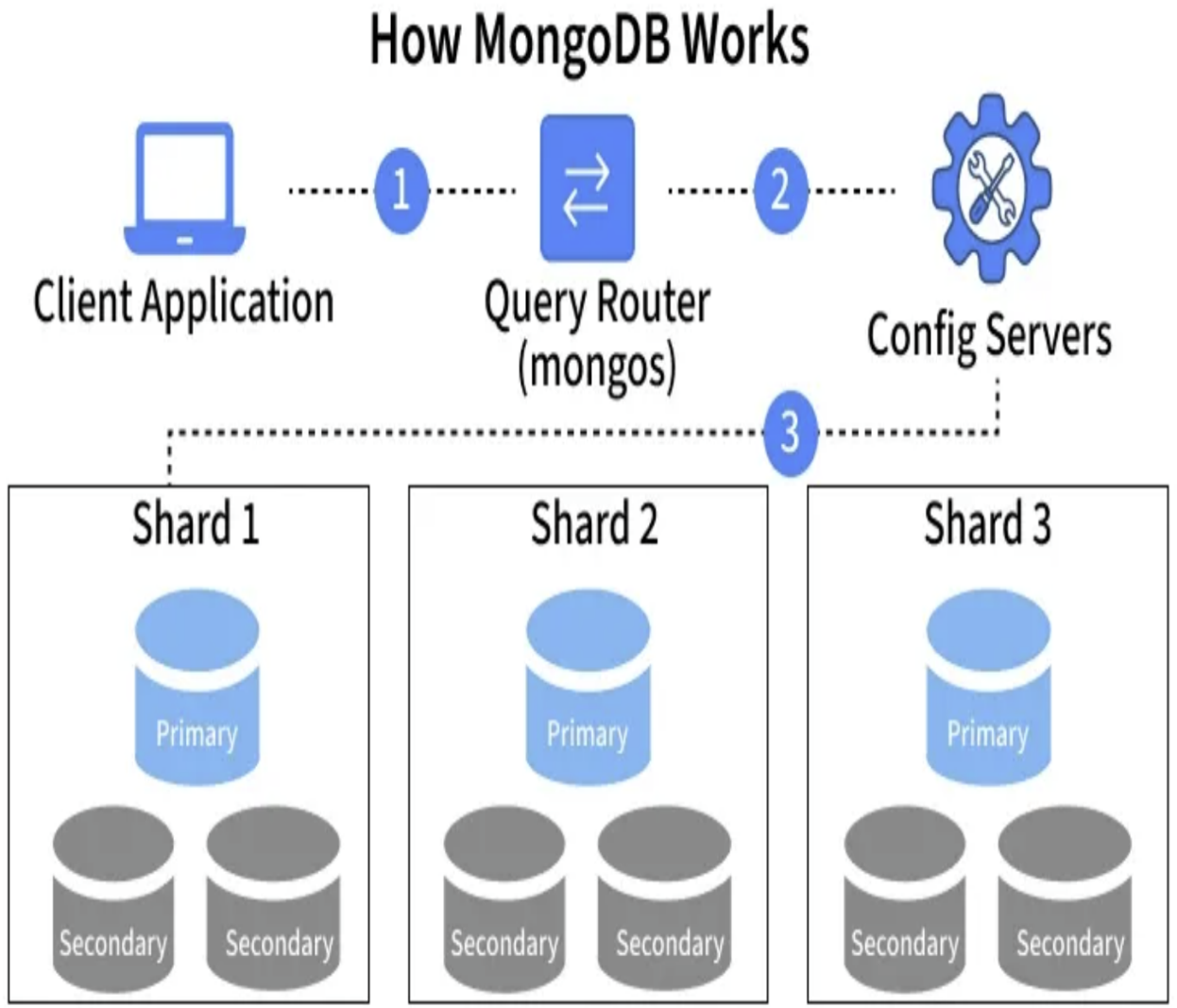
MongoDB Introduction

A simple MongoDB document Structure:

```
{
  "_id": 1,
  "title": “Jungle Book",
  "author": “Rudyard Kipling",
  "url": "https://www.junglebook.org/",
  "tags": ["NoSQL", "Database", "Tutorial"],
  "published": true,
  "views": 1500
}
```

_id: Unique identifier for the document.
title, author, url: Basic fields storing strings.
tags: An array of strings.
published: A Boolean value.
views: A number representing views or count.

Working of MongoDB



In a MongoDB sharded cluster, the components work together as follows:

Client Application: The application sends requests to MongoDB to read or write data.

Query Router (mongos): The query router receives the client request and determines which shard should handle it.

Config Servers: Store metadata about the cluster, including which data resides on which shard.

Shards: Data is horizontally distributed across replica sets, with the primary handling writes and secondaries providing replication and reads.

Primary Node: Handles all write operations.

Secondary Nodes: Replicate data from the primary for high availability and read operations.

Basic SQL Operations

Basic SQL Operations
MongoDB provides several basic operations to manage and manipulate data efficiently:

Create (Insert): Add new documents to a collection.
Read (Find): Retrieve documents from a collection based on query criteria.
Update: Modify existing documents in a collection.
Delete: Remove documents from a collection.

Features of the MongoDB database

MongoDB offers several key features that make it efficient and scalable:

Schema-less Database

Unlike traditional relational databases, MongoDB collections:

- Allow different structures within the same collection.
- Do not require fixed column definitions.
- Enable easy updates and modifications.

Document Oriented: Stores all related data together in a single, flexible document rather than multiple tables.

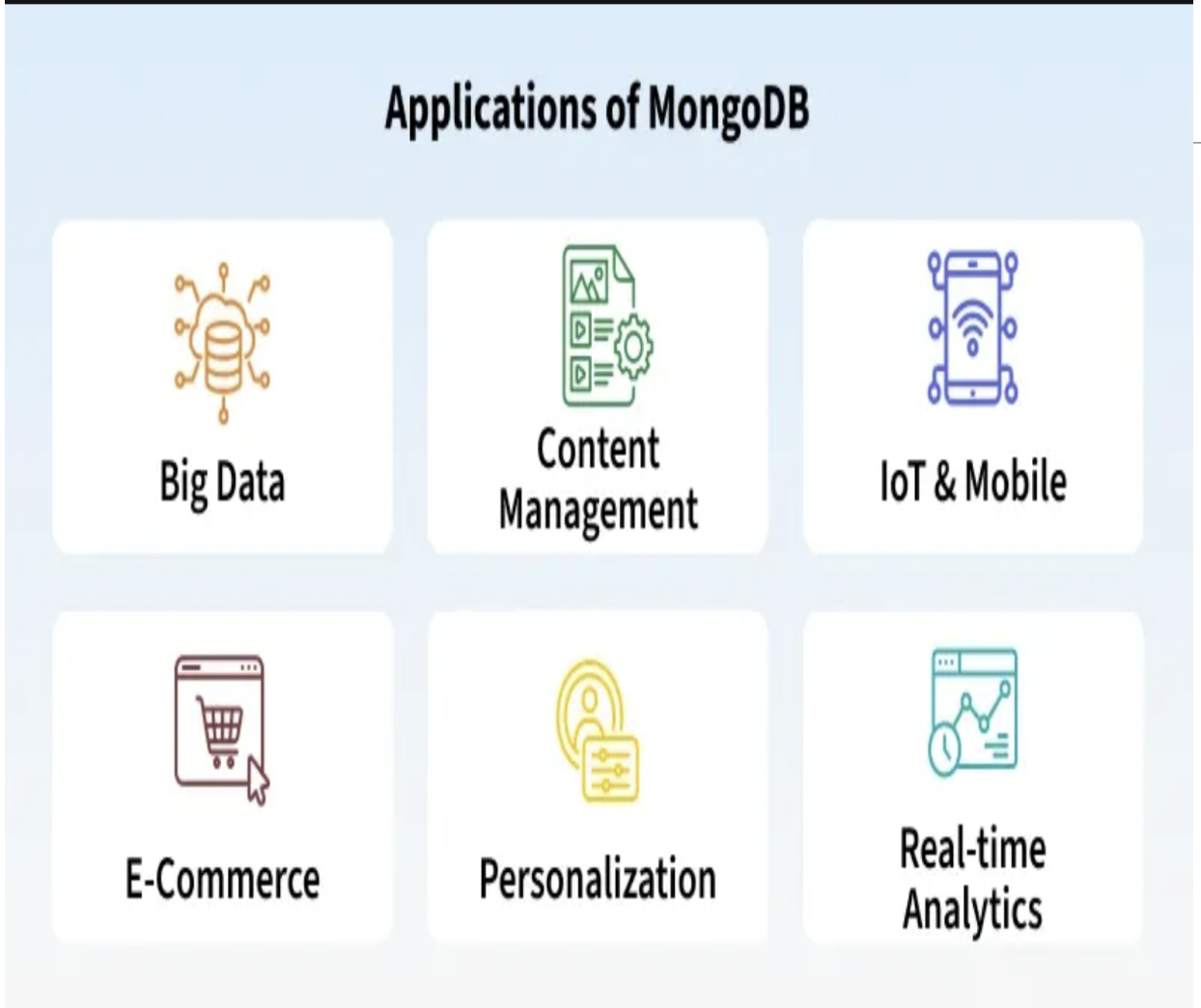
Indexing: Enables fast queries by avoiding full collection scans, crucial for handling large data efficiently.

Scalability: Uses sharding to distribute data across multiple servers; easily add new machines to expand.

Replication and High Availability: Stores multiple data copies on different servers for redundancy and fault tolerance.

Aggregation: Processes and summarizes data (like SQL GROUP BY) using operations such as sum, avg, min, and max.

Real-World Applications of MongoDB



MongoDB powers a wide range of modern applications:

Websites & Apps: Platforms like e-commerce sites use MongoDB to store product catalogs, user profiles, and order histories.

Real-Time Analytics: Companies analyze user activity or sales trends on the fly, such as tracking popular products during a sale.

Content Management & Personalization: Apps manage dynamic content and personalize recommendations, like suggesting movies based on viewing history.

Big Data & IoT: MongoDB handles large volumes of unstructured data from

Language Support by MongoDB

MongoDB offers official drivers for many popular programming languages, enabling easy integration and database operations.

- C & C++:
- Rust
- C#
- Java
- Node.js
- Perl & PHP
- Python
- Ruby & Scala
- Go
- Erlang

Components of MongoDB

MongoDB consists of core components that work together to store, manage, and retrieve data efficiently in a distributed environment.

1. Drivers

MongoDB drivers are client libraries that enable applications to connect to and interact with MongoDB databases using language-specific APIs.

- Provide interfaces and methods for database communication.
- Support multiple programming languages.
- Enable easy integration with applications.
- Popular drivers include C, C++, C#, .NET, Go, Java, Node.js, PHP, Python, Ruby, Scala, Swift, and Mongoid.

Components of MongoDB

2. MongoDB Shell

MongoDB Shell is an interactive JavaScript-based command-line interface used to interact with MongoDB databases.

- Provides a CLI for working with MongoDB.
- Allows running queries and updating data.
- Supports administrative tasks such as backups and restores.

Components of MongoDB

3. Storage Engine

The storage engine is a core MongoDB component responsible for managing how data is stored and accessed in memory and on disk.

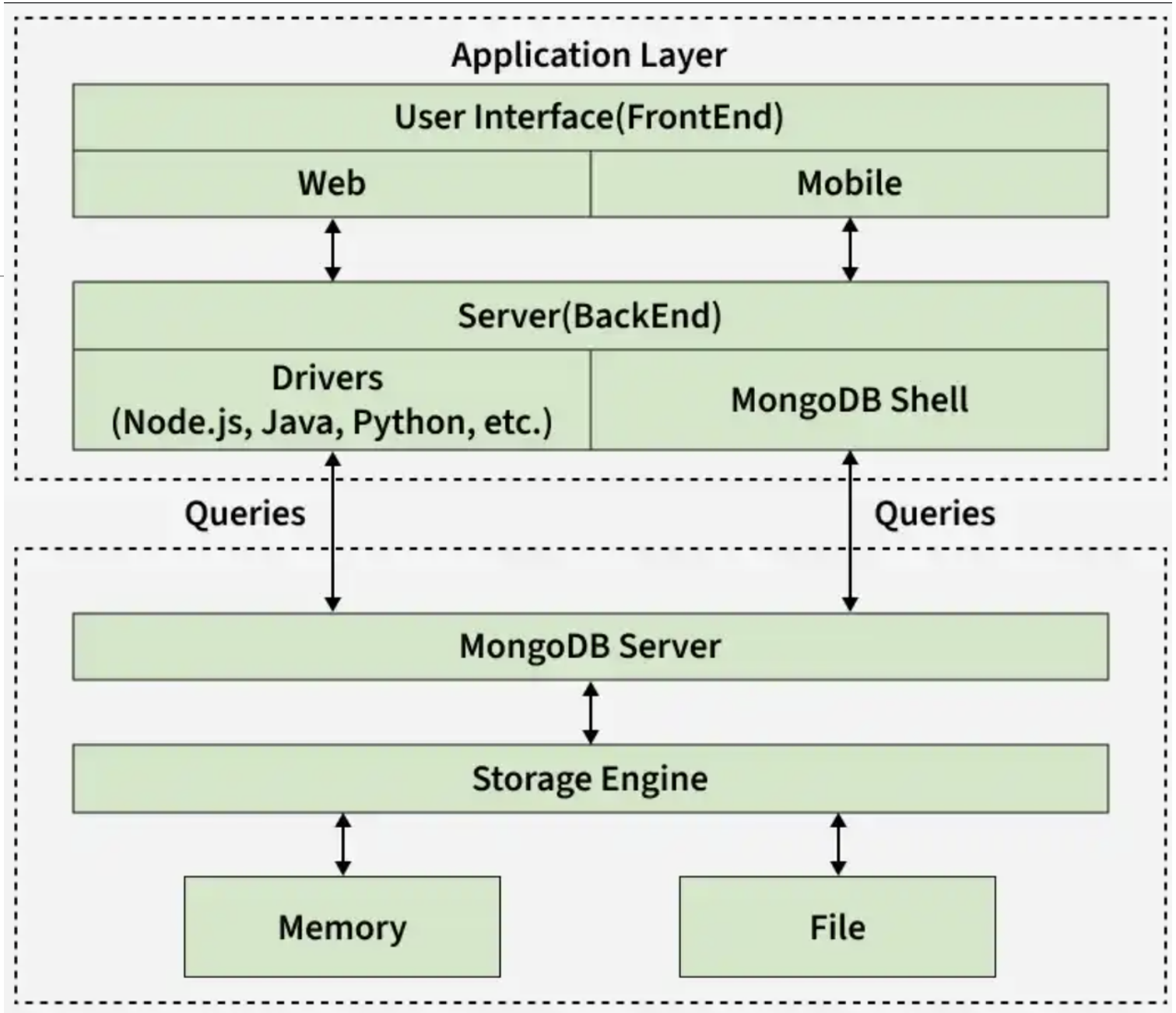
- Controls data storage and retrieval operations.
- Manages reading and writing of data efficiently.
- Allows use of custom storage engines.
- Uses WiredTiger as the default storage engine.

MongoDB can have multiple Storage engines.

Storage Engine	Description	Use Case
WiredTiger	Default storage engine since MongoDB 3.0. It supports document-level concurrency and data compression.	Ideal for high-performance, write-intensive applications.
In-Memory Engine	Stores data in RAM instead of disk, ensuring ultra-fast access.	Suitable for real-time data processing and caching.
MMAPv1	Based on memory-mapped files, designed for read-heavy workloads. It was deprecated after MongoDB 4.0.	Read-heavy applications (use is now discouraged, as it's deprecated).

Working of MongoDB

MongoDB works on Application Layer and the Data Layer. The following image shows the working of MongoDB.



Working of MongoDB

1. Application Layer

The Application Layer, also known as the Final Abstraction Layer, connects users to MongoDB by handling user interaction and server-side processing.

Consists of two parts: Frontend and Backend

Frontend (User Interface): Includes web and mobile applications used by users to interact with MongoDB.

Backend (Server): Handles server-side logic and communicates with MongoDB using drivers or the MongoDB shell.

Frontend sends requests to the backend, which queries the MongoDB server in the Data Layer.

2. Data Layer

The Data Layer is responsible for storing and processing data in MongoDB, handling queries and managing data storage through core components.

MongoDB Server: Receives queries from the application layer and forwards them to the storage engine.

Storage Engine: Handles actual data read and write operations and manages data storage in memory and on disk.

Note: Reading and writing data from memory is much faster than from disk. MongoDB optimizes

MongoDB Data Flow

Outlines the internal flow of data across MongoDB components during operations.

User Interactions: A user interacts with the application (frontend), sends requests (queries).
Backend Processing: The backend (server) receives these requests and communicates with MongoDB using drivers or the Mongo shell.
MongoDB Server: The MongoDB server receives the queries and forwards them to the storage engine.
Storage Engine: The storage engine reads or writes data from/to memory or disk, based on the query.

MongoDB - Terminologies:

A MongoDB Database can be called the container for all the collections.

Database: A container for collections (similar to schema in SQL).

Collection - is a bunch of MongoDB documents. It is similar to tables in RDBMS.
Document - is made of fields. It is similar to a tuple in RDBMS, but it has a dynamic schema here.
[A record stored in JSON-like key-value pairs]

Documents of the same collection **need not have the same set of fields**

MongoDB - Terminologies:

1. Basic Architecture of MongoDB
MongoDB's database structure consists of the following components:

Component	Equivalent in RDBMS	Description
Database	Database	Stores multiple collections.
Collection	Table	Groups related documents.
Document	Row	A BSON object containing key-value pairs.
Field	Column	Stores data attributes within documents.

MongoDB - Working and Features - GeeksforGeeks